

Основные понятия

Методы оптимизации

Александр Богданов

Московский физико-технический институт

11 февраля 2026



Немного истории

- 1847: Коши и градиентный спуск для линейных систем;
- 1950ые: линейное программирование (быстро перешло в нелинейное программирование), появление стохастических методов;
- 1980ые: появление теории для общих задач;
- 2010ые: задачи оптимизации большого размера, теория стохастических методов.

Задача оптимизации

$$\begin{array}{ll}\min_x & f(x) \\ \text{s.t.} & x \in \mathcal{X} \subseteq \mathbb{R}^d \\ & g_i(x) \leq 0, \quad i = \overline{1, n} \\ & h_j(x) = 0, \quad j = \overline{1, m}.\end{array}$$

- $f : \mathcal{X} \rightarrow \mathbb{R}$ — некоторая функция, заданная на множестве Q ;
- $\mathcal{X} \subseteq \mathbb{R}^d$ — подмножество d -мерного пространства;
- $g_i(x)$ и $f_j(x)$ — функции, задающие ограничения.

Задача оптимизации

$$\begin{aligned} \min_x \quad & f(x) \\ \text{s.t.} \quad & x \in \mathcal{X} \subseteq \mathbb{R}^d \\ & g_i(x) \leq 0, \quad i = \overline{1, n} \\ & h_j(x) = 0, \quad j = \overline{1, m}. \end{aligned}$$

- $f : \mathcal{X} \rightarrow \mathbb{R}$ — некоторая функция, заданная на множестве Q ;
- $\mathcal{X} \subseteq \mathbb{R}^d$ — подмножество d -мерного пространства;
- $g_i(x)$ и $h_j(x)$ — функции, задающие ограничения.

Вопрос: Что можно сказать про эту задачу? Сложная ли эта задача?

Задачи оптимизации. Первые наблюдения

- 1 В общем случае задачи оптимизации могут не иметь решения.
Например, задача $\min_{x \in \mathbb{R}} x$ не имеет решения.
- 2 Задачи оптимизации часто нельзя решить аналитически.
- 3 Их сложность зависит от вида целевой функции f , множества $\mathcal{X}$ и может зависеть от размерности x .

Задачи оптимизации. Первые наблюдения

- 1 В общем случае задачи оптимизации могут не иметь решения. Например, задача $\min_{x \in \mathbb{R}} x$ не имеет решения.
- 2 Задачи оптимизации часто нельзя решить аналитически.
- 3 Их сложность зависит от вида целевой функции f , множества $\mathcal{X}$ и может зависеть от размерности x .

Если же задача оптимизации имеет решение, то на практике её обычно решают, вообще говоря, приближённо. Для этого применяются специальные алгоритмы, которые и называют методами оптимизации.

Методы оптимизации

- Нет смысла искать лучший метод для решения конкретной задачи. Например, лучший метод для решений задачи $\min_{x \in \mathbb{R}^d} \|x\|_2^2$ сходится за 1 итерацию: этот метод просто всегда выдаёт ответ $x^* = 0$. Очевидно, что для других задач такой метод не пригоден.
- Эффективность метода определяется для класса задач, так как обычно численные методы разрабатываются для приближённого решения множества однотипных задач.
- Метод разрабатывается для класса задач $\implies$ метод не может иметь с самого начала полной информации о задаче. Вместо этого метод использует модель задачи, например, формулировку задачи, описание функциональных компонент, множества, на котором происходит оптимизация и так далее.

- Предполагается, что численный метод может накапливать специфическую информацию о задаче при помощи некоторого **оракула**. Под оракулом можно понимать некоторое устройство (программу, процедуру), которое отвечает на последовательные вопросы численного метода.

- Предполагается, что численный метод может накапливать специфическую информацию о задаче при помощи некоторого **оракула**. Под оракулом можно понимать некоторое устройство (программу, процедуру), которое отвечает на последовательные вопросы численного метода.

Вопрос: Какого рода вопросы хочется задавать оракулу?

- Предполагается, что численный метод может накапливать специфическую информацию о задаче при помощи некоторого **оракула**. Под оракулом можно понимать некоторое устройство (программу, процедуру), которое отвечает на последовательные вопросы численного метода.

Вопрос: Какого рода вопросы хочется задавать оракулу?

Определение

- **Оракул нулевого порядка** в запрашиваемой точке x возвращает значение целевой функции $f(x)$.
- **Оракул первого порядка** в запрашиваемой точке x возвращает значение целевой функции $f(x)$ и её градиент $\nabla f(x)$.
- **Оракул второго порядка** в запрашиваемой точке x возвращает значение целевой функции $f(x)$, её градиент $\nabla f(x)$ и её гессиан $\nabla^2 f(x)$.

Общая итеративная схема метода оптимизации $\mathcal{M}$

Входные данные: начальная точка x^0 , счётчик итераций $k = 0$,
накапливаемая информационная модель решаемой задачи $I_{-1} = \emptyset$.

Общая итеративная схема метода оптимизации $\mathcal{M}$

Входные данные: начальная точка x^0 , счётчик итераций $k = 0$,
накапливаемая информационная модель решаемой задачи $I_{-1} = \emptyset$.

Основной цикл

- 1 Задать вопрос к оракулу $\mathcal{P}$ в точке x^k .

Общая итеративная схема метода оптимизации $\mathcal{M}$

Входные данные: начальная точка x^0 , счётчик итераций $k = 0$, накапливаемая информационная модель решаемой задачи $I_{-1} = \emptyset$.

Основной цикл

- 1 Задать вопрос к оракулу $\mathcal{P}$ в точке x^k .
- 2 Пересчитать информационную модель: $I_k = I_{k-1} \cup (x^k, \mathcal{P}(x^k))$.

Общая итеративная схема метода оптимизации $\mathcal{M}$

Входные данные: начальная точка x^0 , счётчик итераций $k = 0$, накапливаемая информационная модель решаемой задачи $I_{-1} = \emptyset$.

Основной цикл

- 1 Задать вопрос к оракулу $\mathcal{P}$ в точке x^k .
- 2 Пересчитать информационную модель: $I_k = I_{k-1} \cup (x^k, \mathcal{P}(x^k))$.
- 3 Применить правило метода $\mathcal{M}$ для получения новой точки x^{k+1} по модели I_k .

Общая итеративная схема метода оптимизации $\mathcal{M}$

Входные данные: начальная точка x^0 , счётчик итераций $k = 0$, накапливаемая информационная модель решаемой задачи $I_{-1} = \emptyset$.

Основной цикл

- 1 Задать вопрос к оракулу $\mathcal{P}$ в точке x^k .
- 2 Пересчитать информационную модель: $I_k = I_{k-1} \cup (x^k, \mathcal{P}(x^k))$.
- 3 Применить правило метода $\mathcal{M}$ для получения новой точки x^{k+1} по модели I_k .
- 4 Проверить критерий остановки $\mathcal{T}_\varepsilon$. Если критерий выполнен, то выдать ответ $\bar{x}$, иначе положить $k = k + 1$ и вернуться на шаг 1.

Примеры итерационных методов. Градиентный спуск

Рассмотрим задачу оптимизации

$$\min_{x \in \mathbb{R}^d} f(x),$$

где функция $f(x)$ дифференцируема. Предположим, что в любой точке мы можем посчитать её градиент.

Примеры итерационных методов. Градиентный спуск

Рассмотрим задачу оптимизации

$$\min_{x \in \mathbb{R}^d} f(x),$$

где функция $f(x)$ дифференцируема. Предположим, что в любой точке мы можем посчитать её градиент.

Алгоритм Градиентный спуск с постоянным размером шага

Вход: стартовая точка $x^0 \in \mathbb{R}^d$, размер шага $\gamma > 0$, количество итераций K

- 1: **for** $k = 0, 1, \dots, K - 1$ **do**
- 2: $x^{k+1} = x^k - \gamma \nabla f(x^k)$
- 3: **end for**

Выход: x^K

Примеры итерационных методов. Градиентный спуск

Рассмотрим задачу оптимизации

$$\min_{x \in \mathbb{R}^d} f(x),$$

где функция $f(x)$ дифференцируема. Предположим, что в любой точке мы можем посчитать её градиент.

Алгоритм Градиентный спуск с постоянным размером шага

Вход: стартовая точка $x^0 \in \mathbb{R}^d$, размер шага $\gamma > 0$, количество итераций K

- 1: **for** $k = 0, 1, \dots, K - 1$ **do**
- 2: $x^{k+1} = x^k - \gamma \nabla f(x^k)$
- 3: **end for**

Выход: x^K

Вопрос: чем алгоритм отличается от определения общей итеративной схемы?

Примеры итерационных методов. Градиентный спуск

Рассмотрим задачу оптимизации

$$\min_{x \in \mathbb{R}^d} f(x),$$

где функция $f(x)$ дифференцируема. Предположим, что в любой точке мы можем посчитать её градиент.

Алгоритм Градиентный спуск с постоянным размером шага

Вход: стартовая точка $x^0 \in \mathbb{R}^d$, размер шага $\gamma > 0$, количество итераций K

- 1: **for** $k = 0, 1, \dots, K - 1$ **do**
- 2: $x^{k+1} = x^k - \gamma \nabla f(x^k)$
- 3: **end for**

Выход: x^K

Вопрос: чем алгоритм отличается от определения общей итеративной схемы? В итеративной схеме использовался $\mathcal{T}_\varepsilon$.

Критерии останова

- По аргументу: $\|x^k - x^*\|_2 \leq \varepsilon$.

Вопрос: какие проблемы тут видим?

Критерии останова

- По аргументу: $\|x^k - x^*\|_2 \leq \varepsilon$.

Вопрос: какие проблемы тут видим?

x^* — неизвестно, но можно так:

$$\|x^{k+1} - x^k\|_2 \leq \|x^{k+1} - x^*\|_2 + \|x^k - x^*\|_2.$$

Тогда если $\|x^{k+1} - x^*\|_2 \leq \|x^k - x^*\|_2 \leq \varepsilon$, то $\|x^{k+1} - x^k\|_2 \leq 2\varepsilon$.

Критерии останова

- По аргументу: $\|x^k - x^*\|_2 \leq \varepsilon$.

Вопрос: какие проблемы тут видим?

x^* — неизвестно, но можно так:

$$\|x^{k+1} - x^k\|_2 \leq \|x^{k+1} - x^*\|_2 + \|x^k - x^*\|_2.$$

Тогда если $\|x^{k+1} - x^*\|_2 \leq \|x^k - x^*\|_2 \leq \varepsilon$, то $\|x^{k+1} - x^k\|_2 \leq 2\varepsilon$.

- По функции: $f(x^k) - f^* \leq \varepsilon$.

Критерии останова

- По аргументу: $\|x^k - x^*\|_2 \leq \varepsilon$.

Вопрос: какие проблемы тут видим?

x^* — неизвестно, но можно так:

$$\|x^{k+1} - x^k\|_2 \leq \|x^{k+1} - x^*\|_2 + \|x^k - x^*\|_2.$$

Тогда если $\|x^{k+1} - x^*\|_2 \leq \|x^k - x^*\|_2 \leq \varepsilon$, то $\|x^{k+1} - x^k\|_2 \leq 2\varepsilon$.

- По функции: $f(x^k) - f^* \leq \varepsilon$.

Вопрос: зачем может пригодиться такой критерий?

x^* — не уникально. К тому же часто f^* известно, например, для $f(x) = \|Ax - b\|_2^2$. На практике можно использовать $|f(x^k) - f(x^{k+1})|$.

Критерии останова

- По аргументу: $\|x^k - x^*\|_2 \leq \varepsilon$.

Вопрос: какие проблемы тут видим?

x^* — неизвестно, но можно так:

$$\|x^{k+1} - x^k\|_2 \leq \|x^{k+1} - x^*\|_2 + \|x^k - x^*\|_2.$$

Тогда если $\|x^{k+1} - x^*\|_2 \leq \|x^k - x^*\|_2 \leq \varepsilon$, то $\|x^{k+1} - x^k\|_2 \leq 2\varepsilon$.

- По функции: $f(x^k) - f^* \leq \varepsilon$.

Вопрос: зачем может пригодиться такой критерий?

x^* — не уникально. К тому же часто f^* известно, например, для $f(x) = \|Ax - b\|_2^2$. На практике можно использовать $|f(x^k) - f(x^{k+1})|$.

- По норме градиента: $\|\nabla f(x^k)\|_2 \leq \varepsilon$.

Вопрос: когда такой критерий можно использовать?

Критерии останова

- По аргументу: $\|x^k - x^*\|_2 \leq \varepsilon$.

Вопрос: какие проблемы тут видим?

x^* — неизвестно, но можно так:

$$\|x^{k+1} - x^k\|_2 \leq \|x^{k+1} - x^*\|_2 + \|x^k - x^*\|_2.$$

Тогда если $\|x^{k+1} - x^*\|_2 \leq \|x^k - x^*\|_2 \leq \varepsilon$, то $\|x^{k+1} - x^k\|_2 \leq 2\varepsilon$.

- По функции: $f(x^k) - f^* \leq \varepsilon$.

Вопрос: зачем может пригодиться такой критерий?

x^* — не уникально. К тому же часто f^* известно, например, для $f(x) = \|Ax - b\|_2^2$. На практике можно использовать $|f(x^k) - f(x^{k+1})|$.

- По норме градиента: $\|\nabla f(x^k)\|_2 \leq \varepsilon$.

Вопрос: когда такой критерий можно использовать? В безусловной оптимизации

Сложность методов оптимизации

Определение

- **Аналитическая/Оракульная сложность** — число обращений метода к оракулу, необходимое для решения задачи с точностью ε .
- **Арифметическая/Временная сложность** — общее число вычислений (включая работу оракула), необходимых для решения задачи с точностью ε .
- **Итерационная сложность** — общее число итераций метода, необходимых для решения задачи с точностью ε .

Класс задач минимизации липшицевых функций

$$\min_{x \in [0,1]^d} f(x).$$

- $[0, 1]^d = \{ x \in \mathbb{R}^d \mid 0 \leq x_i \leq 1, i = \overline{1, d} \}$ — кубик;
- Функция $f(x)$ является M -липшицевой на $[0, 1]^d$ относительно ℓ_∞ -нормы:

$$\forall x, y \quad |f(x) - f(y)| \leq M \|x - y\|_\infty = M \max_{i=\overline{1, d}} |x_i - y_i|.$$

Класс задач минимизации липшицевых функций

Наблюдение

Множество $[0, 1]^d$ является ограниченным и замкнутым, то есть компактом, а из липшицевости функции f следует и её непрерывность, поэтому задача имеет решение, ибо непрерывная на компакте функция достигает своих минимального и максимального значений.

- **Цель:** найти $\hat{x} \in [0, 1]^d$: $f(\hat{x}) - f^* \leq \varepsilon$, где f является M -липшицевой на $[0, 1]^d$ относительно ℓ_∞ -нормы.
- **Класс методов:** методы нулевого порядка.

Метод равномерного перебора

Алгоритм Метод равномерного перебора

Require: целочисленный параметр перебора $p \geq 1$

- 1: Сформировать $(p + 1)^d$ точек вида $x_{(i_1, \dots, i_d)} = \left(\frac{i_1}{p}, \frac{i_2}{p}, \dots, \frac{i_d}{p} \right)^\top$, где вектор $(i_1, \dots, i_d) \in \{\overline{0, p}\}^d$
- 2: Среди точек $x_{(i_1, \dots, i_d)}$ найти точку $\hat{x}$ с наименьшим значением целевой функции f .

Ensure: $\hat{x}$

Сходимость метода равномерного перебора

Определение

Пусть задача оптимизации с M -липшицевой целевой функцией f решается с помощью метода равномерного перебора с параметром p . Тогда справедлива следующая оценка сходимости:

$$f(\hat{x}) - f^* \leq \frac{M}{2p},$$

откуда следует, что методу равномерного перебора нужно в худшем случае

$$\left(\left\lfloor \frac{M}{2\varepsilon} \right\rfloor + 2 \right)^d$$

обращений к оракулу, чтобы гарантировать $f(\hat{x}) - f^* \leq \varepsilon$.

Доказательство теоремы. Часть 1

Пусть x^* — решение задачи (возможно, не единственная точка минимума функции f). Тогда в построенной методом сетке из точек найдется такая точка $x_{(i_1, \dots, i_d)}$, что

$$x := x_{(i_1, \dots, i_d)} \preceq x^* \preceq x_{(i_1+1, \dots, i_d+1)} =: x'.$$

Точки x и x' определяют углы кубика сетки, в котором лежит x^* , то есть $x_i^* \in [x_i, x'_i]$ для всех $i = \overline{1, d}$. Отметим, что стороны данного кубика имеют длины $x'_i - x_i = \frac{1}{p}$.

Доказательство теоремы. Часть 2

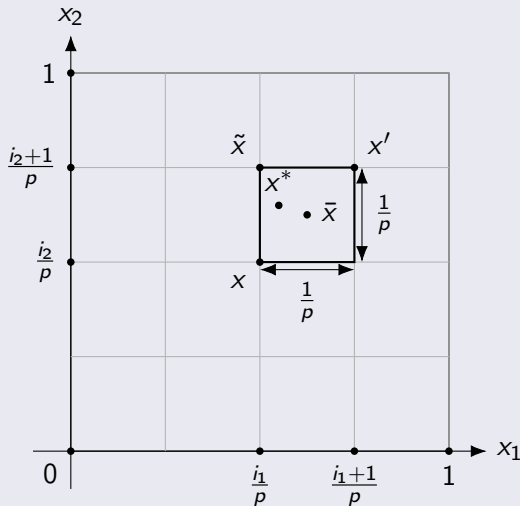
Кроме того, рассмотрим точки $\bar{x}$ и $\tilde{x}$ такие, что $\bar{x} = \frac{x+x'}{2}$ и

$$\tilde{x}_i = \begin{cases} x'_i, & \text{если } x_i^* \geq \bar{x}_i, \\ x_i, & \text{иначе.} \end{cases}$$

$\bar{x}$ — центр кубика, а $\tilde{x}$ определяет ближайший к x^* уголок кубика. Заметим, что $\tilde{x}$ принадлежит сетке и $|\tilde{x}_i - x_i^*| \leq \frac{1}{2p}$, так как в худшем случае x^* расположен в $\bar{x}$, а значит, равноудален от всех уголков. Тогда

$$\|\tilde{x} - x^*\|_\infty = \max_{i \in \overline{1,d}} |\tilde{x}_i - x_i^*| \leq \frac{1}{2p}.$$

Доказательство теоремы. Часть 3



Доказательство теоремы. Часть 4

Поскольку $f(\hat{x}) \leq f(\tilde{x})$ (так как $f(\hat{x})$ — минимум по всем точкам сетки), получаем

$$f(\hat{x}) - f^* \leq f(\tilde{x}) - f^* \leq M \|\tilde{x} - x^*\|_\infty \leq \frac{M}{2p}.$$

Выписанная выше оценка достигается методом равномерного перебора за $(p+1)^d$ обращений к оракулу. Ограничим сверху значением ε :

$$f(\hat{x}) - f^* \leq \frac{M}{2p} \leq \varepsilon \implies p \geq \frac{M}{2\varepsilon}.$$

Возьмем $p = \lfloor \frac{M}{2\varepsilon} \rfloor + 1$, тогда метод сделает $(\lfloor \frac{M}{2\varepsilon} \rfloor + 2)^d$ обращений к оракулу нулевого порядка и выдаст ответ с ε -точностью.

Сложность задач оптимизации

Вопрос: хороший результат получили или нет?

Сложность задач оптимизации

Вопрос: хороший результат получили или нет?

- Предположим, $M = 2$, $d = 13$ и $\varepsilon = 0.01$, то есть размерность задачи сравнительно небольшая (можно сказать, что никакая для задач оптимизации в современных приложениях) и точность решения задачи не слишком высокая.

Сложность задач оптимизации

Вопрос: хороший результат получили или нет?

- Предположим, $M = 2$, $d = 13$ и $\varepsilon = 0.01$, то есть размерность задачи сравнительно небольшая (можно сказать, что никакая для задач оптимизации в современных приложениях) и точность решения задачи не слишком высокая.
- Необходимое число обращений к оракулу:
$$\left(\left\lfloor \frac{M}{2\varepsilon} \right\rfloor + 2\right)^d = 102^{13} > 10^{26}.$$

Сложность задач оптимизации

Вопрос: хороший результат получили или нет?

- Предположим, $M = 2$, $d = 13$ и $\varepsilon = 0.01$, то есть размерность задачи сравнительно небольшая (можно сказать, что никакая для задач оптимизации в современных приложениях) и точность решения задачи не слишком высокая.
- Необходимое число обращений к оракулу:

$$\left(\left\lfloor \frac{M}{2\varepsilon} \right\rfloor + 2\right)^d = 102^{13} > 10^{26}.$$
- Сложность одного вызова оракула не менее 1 арифметической операции (FLOP).

Сложность задач оптимизации

Вопрос: хороший результат получили или нет?

- Предположим, $M = 2$, $d = 13$ и $\varepsilon = 0.01$, то есть размерность задачи сравнительно небольшая (можно сказать, что никакая для задач оптимизации в современных приложениях) и точность решения задачи не слишком высокая.
- Необходимое число обращений к оракулу:
 $(\lfloor \frac{M}{2\varepsilon} \rfloor + 2)^d = 102^{13} > 10^{26}$.
- Сложность одного вызова оракула не менее 1 арифметической операции (FLOP).
- Производительность современной видеокарты порядка 10^{14} арифметических операций в секунду (82 TFLOPs на NVIDIA GeForce RTX 4090).

Сложность задач оптимизации

Вопрос: хороший результат получили или нет?

- Предположим, $M = 2$, $d = 13$ и $\varepsilon = 0.01$, то есть размерность задачи сравнительно небольшая (можно сказать, что никакая для задач оптимизации в современных приложениях) и точность решения задачи не слишком высокая.
- Необходимое число обращений к оракулу:
$$\left(\left\lfloor \frac{M}{2\varepsilon} \right\rfloor + 2\right)^d = 102^{13} > 10^{26}.$$
- Сложность одного вызова оракула не менее 1 арифметической операции (FLOP).
- Производительность современной видеокарты порядка 10^{14} арифметических операций в секунду (82 TFLOPs на NVIDIA GeForce RTX 4090).
- Общее время: хотя бы 10^{12} секунд, что займёт порядка 30 тысяч лет.

Верхние и нижние оценки

- **Вопрос:** что мы сейчас получили? верхнюю или нижнюю оценку?
что такое верхняя оценка?

Верхние и нижние оценки

- **Вопрос:** что мы сейчас получили? верхнюю или нижнюю оценку? что такое верхняя оценка?
- **Верхняя оценка** — гарантия, что рассматриваемый метод из класса методов на любой задаче из класса решаемых задач имеет оракульную/арифметическую/итерационную сложность не хуже, чем утверждает верхняя оценка.
- **Нижняя оценка** — гарантия, что для любого метода из класса методов существует «плохая» задача из класса решаемых задач, такая, что метод имеет оракульную/арифметическую/итерационную сложность не лучше, чем утверждает нижняя оценка.
- Возникает вопрос: может мы плохо вывели верхнюю оценку (неидеальный анализ), может ли предложить другой метод из рассматриваемого класса, который будет находить приближённое решение существенно быстрее?

Нижние оценки для методов нулевого порядка

Теорема

Пусть задача оптимизации с M -липшицевой целевой функцией f решается с помощью методов нулевого порядка с точностью $\varepsilon < \frac{M}{2}$. Тогда необходимо не менее

$$\left(\left\lfloor \frac{M}{2\varepsilon} \right\rfloor - 1 \right)^d - 1$$

обращений к оракулу, чтобы гарантировать $f(\hat{x}) - f^* \leq \varepsilon$.

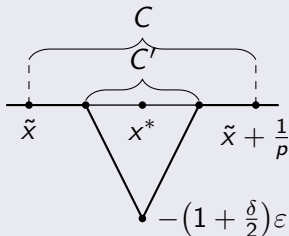
Идея доказательства теоремы

Предположим, что существует такой метод, который решает задачу за меньшее количество обращений к оракулу с точностью ε (по функции).

Опровергнем это, построив такую функцию, на которой метод не сможет найти ε -решение, при помощи сопротивляющегося оракула.

Пусть изначально наша целевая функция $f(x)$ всюду равна 0.

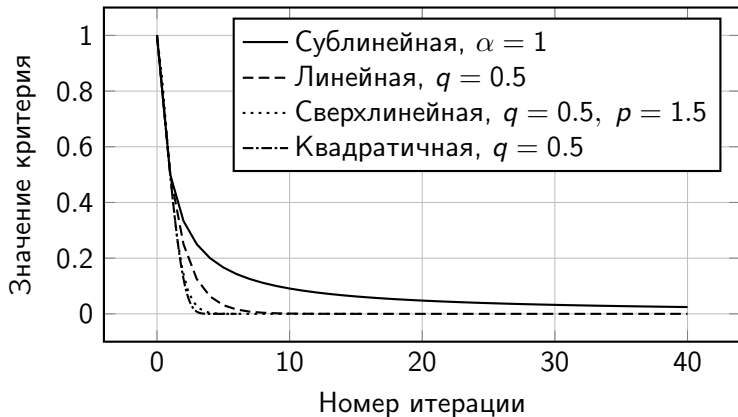
Запустим метод, он запросит значение f в N точках, везде получит 0 и выдаст какую-то точку как ответ.



Скорости сходимости

- Сублинейная: $\|x^k - x^*\|_2 \leq \frac{C}{k^\alpha}$, $C > 0$, $\alpha > 0$.
- Линейная: $\|x^k - x^*\|_2 \leq Cq^k$, $C > 0$, $0 < q < 1$.
- Сверхлинейная: $\|x^k - x^*\|_2 \leq Cq^{k^p}$, $C > 0$, $0 < q < 1$, $p > 1$.
- Квадратичная: $\|x^k - x^*\|_2 \leq Cq^{2^k}$, $C > 0$, $0 < q < 1$.
Квадратичная локальная: $\|x^{k+1} - x^*\|_2 \leq C\|x^k - x^*\|_2^2$, $C > 0$.

Скорости сходимости



Скорости сходимости

